

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Mauricio Cruz Quijada

11 de septiembre de 2026

20:03

1. Introducción

En esta tarea se comparan cuatro algoritmos de ordenamiento y dos de multiplicación de matrices. El objetivo es observar cómo cambian sus tiempos y su uso de memoria al variar el tamaño y las características de la entrada, y relacionar esas diferencias con la complejidad teórica y con el código usado. La comparación corresponde a estas implementaciones y a los casos del enunciado [2]; no busca afirmar que un algoritmo sea siempre mejor.

El código fuente está disponible en <https://github.com/malishoU/Tarea1-Algoco>.

2. Entorno experimental

Los experimentos se realizaron en un equipo con procesador Intel Core i5-12500H, 16 procesadores lógicos y aproximadamente 7,6 GiB de RAM disponibles para Ubuntu 24.04.4 LTS en WSL2. Se utilizó C++17 con g++ 13.3.0 y optimización -O2. Los datos se generaron con Python 3.12.3 y NumPy 1.26.4; los gráficos se prepararon con Matplotlib 3.11.1 [8].

En ordenamiento se probaron tamaños 10 , 10^3 , 10^5 y 10^7 , entradas ascendentes, descendentes y aleatorias, con valores D1 (0 a 9) y D7 (0 a 10^7 , incluidos). En matrices se probaron tamaños 16, 64, 256 y 1024, tipos denso, diagonal y disperso, y dominios D0 (0 y 1) y D10 (0 a 9). Cada configuración tiene tres muestras: a, b y c. Las matrices dispersas se generan intentando llenar un 10% de las posiciones; como pueden repetirse posiciones, la proporción final de no ceros puede ser menor. Las matrices se almacenan completas, incluso cuando contienen muchos ceros.

Se fijó la semilla 20260911 al iniciar cada generador; en matrices se fijó tanto en NumPy como en random. Los datos se repiten ejecutando los scripts completos en el mismo orden y con las mismas versiones. Todos los resultados de este informe corresponden a la nueva corrida con estas semillas. Se obtuvieron 288 registros de ordenamiento y 144 de matrices, sin casos faltantes ni duplicados. Los outputs guardados se contrastaron con un ordenamiento de referencia y con el producto matricial calculado con NumPy. Esta comprobación valida los archivos guardados; no valida resultados que no llegaron a generarse por un timeout.

El tiempo se midió con el reloj `steady_clock` de `std::chrono`, alrededor de la llamada al algoritmo, sin incluir lectura ni escritura de archivos. La memoria es el máximo residente del proceso indicado por `getrusage`, en KiB; incluye las entradas y no equivale solo a la memoria auxiliar del algoritmo. Cada ejecución se inicia en un proceso distinto. Las funciones de ordenamiento retornan el vector por valor, por lo que la medición también puede incluir el costo de esa interfaz.

Se usaron límites de 30 s para ordenamiento y 60 s para matrices mediante `timeout`. Estos límites se aplican al proceso completo, incluida la entrada/salida, y no son tiempos exactos del algoritmo. Hubo 11 timeouts en ordenamiento y 18 en matrices. Se presenta la media de las muestras que terminaron y el rango mínimo–máximo en los gráficos; los timeouts se señalan aparte y nunca se sustituyen por cero. Hay tres entradas por configuración, no tres repeticiones de una misma entrada. No se realizaron pruebas de significancia ni se midieron por separado comparaciones, copias o asignaciones.

3. Resultados y Análisis

3.1. Ordenamiento de arreglos

La Tabla 1 compara todas las clases de entrada para $n = 10^7$. Las figuras y los CSV permiten revisar también los tamaños menores. TO significa que ninguna de las tres muestras terminó dentro del límite.

Entrada	Dominio	Merge	Quick	Patience	<code>std::sort</code>
Aleatorio	D1	0.673	TO	1.712	0.187
Aleatorio	D7	1.240	0.742	4.309	0.653
Ascendente	D1	0.464	TO	1.484	0.083
Ascendente	D7	0.457	0.175	15.979*	0.119
Descendente	D1	0.478	TO	0.681	0.084
Descendente	D7	0.498	0.236	0.666	0.107

Cuadro 1: Tiempo medio en segundos para $n = 10^7$. *PatienceSort ascendente D7: una muestra completada y dos timeouts; no es una media de tres éxitos.

3.1.1. MergeSort

MergeSort divide el arreglo en mitades y después mezcla las partes ordenadas [4]. La división tiene aproximadamente $\log_2 n$ niveles; en cada nivel de mezcla se procesa una cantidad total de elementos pro-

porcional a n . Esto explica el costo $O(n \log n)$ [3]. Nuestro código crea vectores auxiliares y copia elementos en cada mezcla. También realiza la división y la mezcla si la entrada ya está ordenada: no incorpora una comprobación para saltarse las mezclas que no hagan falta.

Para $n = 10^7$, los promedios fueron de 0,457 a 1,240 s según la configuración. Las entradas aleatorias D7 fueron más lentas que las ascendentes D7 (1,240 frente a 0,457 s). Por lo tanto, compartir el mismo orden de complejidad no implica tener el mismo tiempo para todas las entradas. Su memoria máxima rondó los 98,7 MiB en ese tamaño, frente a unos 79,8 MiB de `std::sort`. Los vectores auxiliares son una explicación coherente con esta diferencia, aunque no se midió por separado el costo de cada copia.

3.1.2. QuickSort

La implementación usa el elemento central como pivote y una partición de Lomuto [5]. Cada partición recorre el segmento actual. Si las divisiones están equilibradas, el costo esperado es $O(n \log n)$; si quedan muy desbalanceadas, puede llegar a $O(n^2)$ [3]. Por ejemplo, con todos los valores iguales, la condición $\leq$ deja casi todos los elementos al mismo lado. El trabajo pasa a ser proporcional a $(n - 1) + (n - 2) + \dots + 1$.

En D1 solo existen diez valores posibles, por lo que las entradas grandes contienen muchas repeticiones. Las nueve pruebas de $n = 10^7$ en D1 terminaron en timeout. En D7 sí terminaron las nueve, con medias de 0,742 s en aleatorio, 0,175 s en ascendente y 0,236 s en descendente. Este contraste es consistente con el efecto de los duplicados en la partición, pero no contamos las particiones ni demostramos experimentalmente que fueran la única causa. Elegir un pivote central no resuelve por sí solo ese problema.

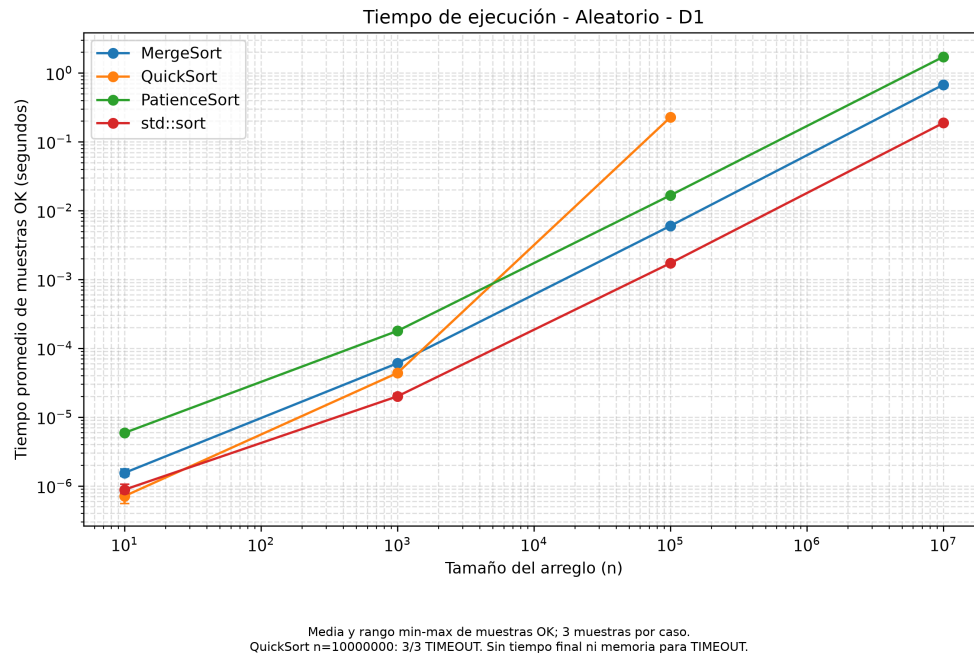


Figura 1: Tiempo en entradas aleatorias D1. La nota identifica los tres timeouts de QuickSort en el tamaño mayor.

3.1.3. PatienceSort

PatienceSort distribuye los valores en pilas y luego los extrae mediante un heap [9]. Nuestro código busca la pila con búsqueda binaria. Con k pilas, esa búsqueda tiene costo $O(\log k)$; las operaciones del heap también dependen de k . La organización habitual permite un costo $O(n \log n)$, pero el número de pilas y sus estructuras auxiliares influyen en el costo concreto de esta implementación.

En una entrada ascendente con muchos valores distintos se forman muchas pilas. En D7 y $n = 10^7$ hubo dos timeouts y una muestra completada en 15,979 s, con 5,42 GiB de memoria máxima. En descendente D7 la media fue de 0,666 s y aproximadamente 82,0 MiB. Es una diferencia grande para el mismo tamaño. La cantidad de pilas y las asignaciones asociadas ofrecen una explicación plausible, sin que hayamos medido ese costo de forma aislada. El punto de 15,979 s representa solo el caso terminado; omitir los otros dos resultados haría parecer más favorable el comportamiento de esta configuración.

3.1.4. `std::sort`

Se utilizó directamente la función de la biblioteca estándar [1], cuya garantía de comparaciones es $O(n \log n)$. Para $n = 10^7$ tuvo la menor media entre las ejecuciones completadas en las seis configuraciones: de 0,083 a 0,653 s. En aleatorio D1 fue de 0,187 s, frente a 0,673 s de MergeSort y 1,712 s de PatienceSort. En aleatorio D7 la distancia con QuickSort fue menor: 0,653 frente a 0,742 s.

Estos resultados muestran que la notación asintótica no distingue todos los costos de una implementación. No medimos el número de operaciones internas de `std::sort`, por lo que no atribuimos su ventaja a una única optimización. Tampoco generalizamos el orden observado a otros equipos o bibliotecas. En los tamaños mínimos, los tiempos son muy pequeños y una única ejecución por muestra no basta para interpretar diferencias pequeñas.

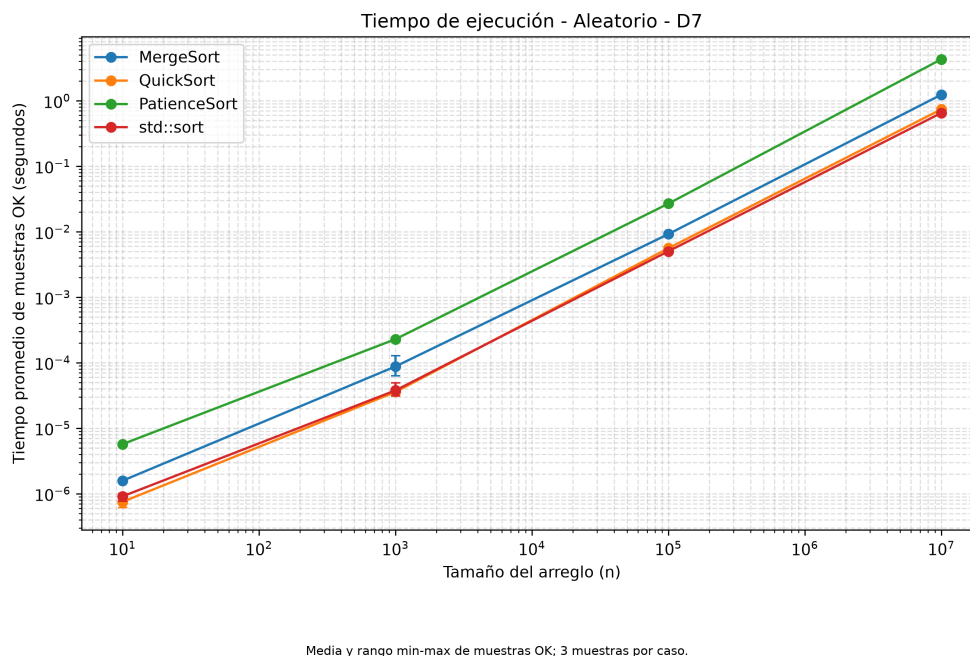


Figura 2: Tiempo en entradas aleatorias D7. Las barras muestran el rango de las tres muestras, no intervalos de confianza.

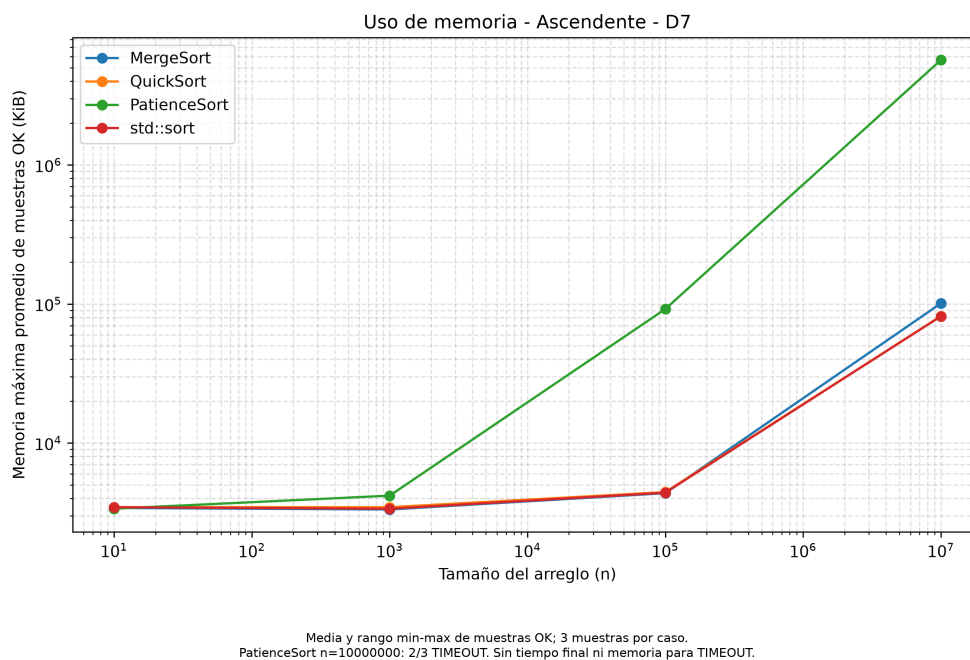


Figura 3: Memoria máxima del proceso en ascendente D7, con ambos ejes logarítmicos. El dato grande de PatienceSort corresponde solo a una muestra completada.

3.2. Multiplicación de matrices

3.2.1. Naive

Naive calcula cada una de las n^2 posiciones del resultado con un recorrido de longitud n : realiza n^3 multiplicaciones escalares [6]. Los tres ciclos se ejecutan también sobre los ceros; el código no aprovecha de forma especial las matrices diagonales ni las dispersas. Por eso, tener menos valores no nulos no elimina esos recorridos ni convierte esta implementación en un algoritmo para matrices dispersas.

n	Naive (s)	Strassen (s)
16	$3,972 \times 10^{-6}$	0,00107331
64	0,000117777	0,0493432
256	0,00650711	2,35752
1024	1,12541	TO (3/3)

Cuadro 2: Tiempo medio para matrices diagonales D0. Los valores se calcularon con los CSV de la corrida final. TO indica timeout, no un tiempo medido.

3.2.2. Strassen

Strassen usa siete multiplicaciones recursivas de submatrices en lugar de ocho, a cambio de sumas y restas adicionales [7]. La recurrencia $T(n) = 7T(n/2) + O(n^2)$ da un costo $O(n^{\log_2 7}) \approx O(n^{2,807})$, mejor asintóticamente que el costo cúbico de Naive [3]. Esto describe el crecimiento, pero no garantiza un menor tiempo para cualquier tamaño.

Nuestra implementación copia matrices, construye submatrices y realiza operaciones auxiliares en cada nivel. Además, llega hasta matrices de 1×1 , sin cambiar a Naive para los casos pequeños. Esos costos son una explicación consistente con el resultado observado: Strassen fue más lento en todas las configuraciones donde ambos terminaron y alcanzó el límite en las 18 pruebas de $n = 1024$. No se midió por separado cuánto corresponde a copias, recursión o asignaciones, ni se encontró un tamaño de cruce donde Strassen gane. Usar Naive como caso base sería un experimento futuro, no una mejora evaluada en esta entrega.

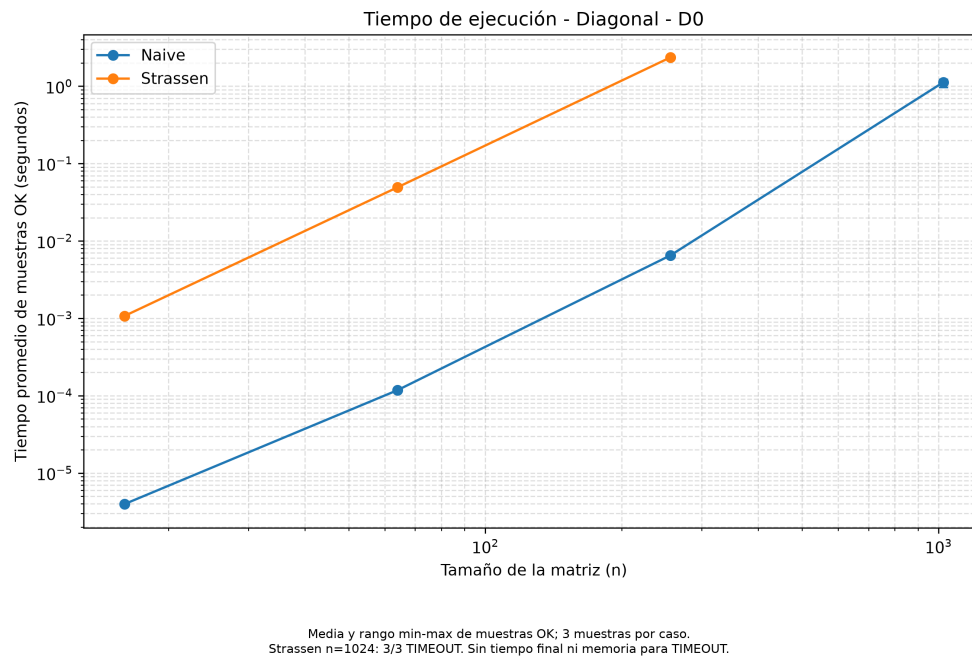


Figura 4: Tiempo de matrices diagonales D0. Strassen no tiene un tiempo final registrado en 1024; las tres muestras llegaron al límite del proceso de 60 s.

3.2.3. Comparación entre entradas y memoria

Tipo	Dominio	Media Naive (s)	Rango Naive (s)
Densa	D0	0.894	0.891–0.899
Densa	D10	0.909	0.897–0.930
Diagonal	D0	1.125	0.958–1.226
Diagonal	D10	0.903	0.892–0.918
Dispersa	D0	0.880	0.872–0.889
Dispersa	D10	0.903	0.883–0.942

Cuadro 3: Naive en $n = 1024$: media y rango de tres muestras por configuración. Strassen tuvo tres timeouts en cada fila.

La Tabla 3 evita concluir que los tipos de entrada sean idénticos en tiempo. La media de diagonal D0 fue mayor que las otras, pero las tres muestras no permiten separar el efecto de la entrada de la variación del sistema. El hecho comprobable en el código es que los ciclos de Naive no omiten ceros.

Naive usó en promedio 4096 KiB en matrices dispersas D10 de tamaño 256. Strassen usó en promedio 7552 KiB en matrices dispersas D10 de tamaño 256. La diferencia es coherente con las matrices auxiliares de Strassen. La medida incluye el proceso completo y no permite atribuir toda la diferencia a una estructura concreta. En los casos de timeout no se registró memoria; por lo tanto, no se puede comparar su consumo final con el de Naive.

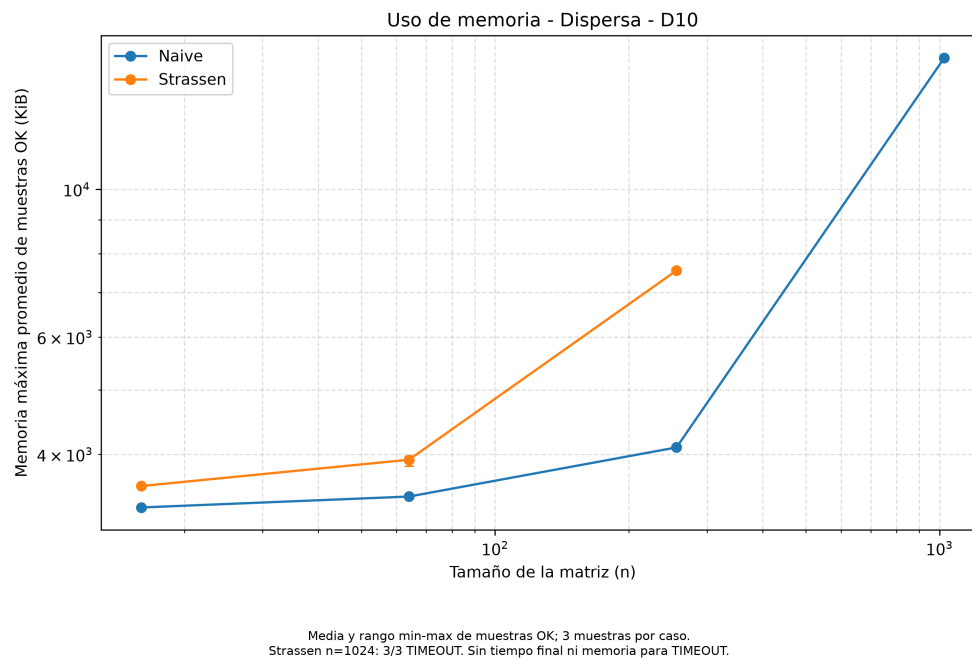


Figura 5: Memoria máxima del proceso en matrices dispersas D10. La serie de Strassen no incluye 1024 porque las ejecuciones terminaron en timeout.

4. Conclusiones

En esta corrida, `std::sort` tuvo las menores medias de tiempo para las seis configuraciones de arreglos de 10^7 elementos. MergeSort mantuvo un comportamiento más regular que QuickSort, mientras que QuickSort alcanzó el límite en las nueve entradas grandes de D1. PatienceSort mostró una fuerte dependencia del tipo de entrada: en ascendente D7 hubo dos timeouts y una muestra terminada con un consumo de memoria mucho mayor que los otros algoritmos.

En matrices, Naive fue más rápido en los casos comparables y completó todas las pruebas. Strassen alcanzó el límite en las 18 pruebas de tamaño 1024. Su mejor complejidad asintótica no compensó los costos de esta implementación en los tamaños evaluados. No se concluye que Naive siempre sea superior, sino que fue más conveniente en este experimento.

La complejidad teórica ayuda a interpretar el crecimiento, pero también importan los datos y la forma de implementar las operaciones. Las explicaciones sobre particiones, pilas y copias se apoyan en el código y son consistentes con las mediciones; no sustituyen una medición aislada de cada causa. Como continuación, se podría comparar una partición de QuickSort que agrupe los iguales y una versión de Strassen con Naive como caso base. Estas variantes no se probaron aquí.

Referencias

- [1] cppreference.com. *std::sort*. URL: <https://en.cppreference.com/w/cpp/algorithm/sort>.
- [2] Equipo de Algoritmos y Complejidad. *Tarea 1: enunciado y material de apoyo*. Asignatura INF-221, Universidad Técnica Federico Santa María. Material proporcionado mediante el aula virtual. 2026.
- [3] Jeff Erickson. *Algorithms*. Jun. de 2019. ISBN: 978-1-792-64483-2.
- [4] GeeksforGeeks. *C++ Program For Merge Sort*. URL: <https://www.geeksforgeeks.org/cpp/cpp-program-for-merge-sort/>.
- [5] GeeksforGeeks. *C++ Program For QuickSort*. URL: <https://www.geeksforgeeks.org/cpp/cpp-program-for-quicksort/>.
- [6] GeeksforGeeks. *C++ Program to Multiply Two Matrices*. URL: <https://www.geeksforgeeks.org/cpp/cpp-program-to-multiply-two-matrices/>.
- [7] GeeksforGeeks. *Strassen's Matrix Multiplication Algorithm*. URL: <https://www.geeksforgeeks.org/strassens-matrix-multiplication-algorithm-implementation/>.
- [8] Matplotlib. *Matplotlib*. URL: <https://matplotlib.org/>.
- [9] Rosetta Code. *Sorting algorithms/Patience sort*. Implementación de la sección C++. URL: https://rosettacode.org/wiki/Sorting_algorithms/Patience_sort.